SMART RECRUITMENT MATCHING PLATFORM

Full Technical Stack Document

Project Type	Secure web-based recruitment matching platform
Implementation	ASP.NET Core Web API with integrated static frontend
Target Framework	.NET 8.0 (net8.0)
Document Basis	Verified from the submitted final project source code (finished.zip)

1. Purpose

This document records the technologies actually used in the final Smart Recruitment Matching Platform project. The stack below was derived from the submitted project files, including the .csproj configuration, Program.cs, appsettings.json, backend source structure, and the HTML/CSS/JavaScript frontend under wwwroot.

2. Core Technical Stack

Area	Technology Used	Evidence in Final Project
.NET Version	.NET 8.0 / TargetFramework net8.0	SmartRecruitment-Project.csproj targets net8.0.
Frontend	HTML5, CSS3, Vanilla JavaScript	Static pages, CSS stylesheets and JavaScript modules are stored under wwwroot.
Backend	ASP.NET Core Web API	Project uses Microsoft.NET.Sdk.Web, controllers, middleware, services and repositories.
Database	Microsoft SQL Server LocalDB	Connection string uses (localdb)\MSSQLLocalDB with database SmartRecruitmentDB.
ORM / Data Access	Entity Framework Core 8.0.22	SQL Server provider, migrations, DbContext, repositories and LINQ-based data access.
Authentication	JWT Bearer Authentication	JWT issuer, audience, key and lifetime validation configured in Program.cs.
API Documentation / Testing	Swagger / OpenAPI via Swashbuckle.AspNetCore 6.6.2	Swagger generator and Bearer security definition configured for development.

3. Programming Languages

- C# - backend API, business logic, matching engine, repositories, services, controllers and middleware.
- JavaScript - frontend API calls, authentication/session handling, validation and page interactions.
- HTML5 - separate user-interface pages for authentication, Job Seeker, Employer and Administrator workflows.
- CSS3 - styling, responsive layouts, components, dashboards, jobs, profiles and authentication screens.
- SQL - database operations are produced through Entity Framework Core and SQL Server migrations.

4. Backend Framework and Architecture

The backend is an ASP.NET Core Web API application using a layered architecture. The final project contains Controllers, Services, Repositories, Interfaces, DTOs, Models, Options, Middleware, Data and Migrations. Dependency Injection is used to connect repository and service interfaces to their implementations.

- ASP.NET Core Controllers for REST API endpoints.
- Service layer for business rules and workflow logic.
- Repository layer for Entity Framework Core data access.
- DTOs for request and response models.
- GlobalExceptionMiddleware for centralized exception handling.
- Built-in ASP.NET Core Dependency Injection for service registration.
- CORS policy configured for frontend development.
- HTTPS redirection, authentication and authorization middleware in the request pipeline.

5. Database and Persistence Technologies

- Microsoft SQL Server LocalDB is configured as the development database through the DefaultConnection connection string.
- Entity Framework Core SQL Server provider version 8.0.22 is used for database access.
- Entity Framework Core migrations are included, including the initial database migration and a unique application constraint migration.
- DbContext is registered using UseSqlServer in Program.cs.
- Repository classes encapsulate database operations for authentication, profiles, jobs, applications, notifications, contact requests and administration.

6. APIs and Services Used

API / Service	Purpose
RESTful HTTP API	Frontend communicates with backend endpoints using GET, POST, PUT and PATCH requests.

JWT Token Service	Generates and validates tokens used to secure protected endpoints and role-specific operations.
Job Seeker Service	Profile management, skills and secure CV operations.
Employer / Job Service	Employer profile and vacancy management.
Matching Service	Deterministic matching of candidate and vacancy data.
Job Discovery Service	Open-vacancy discovery and job matching information.
Application Service	Application creation, tracking, applicant ranking and status updates.
Contact Request Service	Employer-to-candidate contact request workflow.
Notification Service	Database-backed in-application notifications.
Admin Service	Administrator dashboard and account management.
Local File Storage Service	Stores uploaded CV files in a protected server-side folder.

7. Security Technologies

- Microsoft.AspNetCore.Authentication.JwtBearer 8.0.22 for JWT-based authentication.
- Microsoft.IdentityModel.Tokens for issuer, audience, lifetime and signing-key validation.
- Role-based authorization for JobSeeker, Employer and Administrator workflows.
- Password hashing is implemented in the authentication workflow; plain-text passwords are not intended for storage.
- CV files are stored under ProtectedStorage/CVs, outside the public wwwroot folder.
- Global exception middleware prevents normal API responses from exposing unhandled application failures directly.

8. Frontend Technologies

The final project does not use React, Angular, Vue, Bootstrap or another JavaScript frontend framework. It uses a custom frontend built with standard web technologies and is served by ASP.NET Core as static files.

- HTML5 pages for login and registration.
- Separate Job Seeker pages for dashboard, profile, jobs, job details, applications, contact requests and notifications.
- Separate Employer pages for dashboard, profile, vacancies, vacancy creation/editing, applicants, contact requests and notifications.
- Administrator pages for dashboard and user management.
- CSS files for reset, variables, global styles, components, layouts, authentication, dashboards, jobs, profiles and responsive design.
- Vanilla JavaScript modules use the browser Fetch API to call backend endpoints.
- JWT is attached to API calls using the Authorization: Bearer <token> header.
- FormData is used for file-upload requests such as CV upload.

9. Matching Technology

The project uses a deterministic, rule-based matching engine written in C#. It does not use AI or machine-learning services. The configured default weighting is:

Matching Component	Weight
Skills	60%
Experience	20%
Education	10%
Location	10%

The MatchingOptions class validates that all weights are non-negative and that their total equals exactly 100%.

10. NuGet Packages and Libraries

Package	Version	Usage
Microsoft.AspNetCore.Authentication.JwtBearer	8.0.22	JWT Bearer authentication.
Microsoft.EntityFrameworkCore.Design	8.0.22	EF Core design-time tooling and migrations support.
Microsoft.EntityFrameworkCore.SqlServer	8.0.22	SQL Server database provider.
Microsoft.EntityFrameworkCore.Tools	8.0.22	Entity Framework Core tooling.
Swashbuckle.AspNetCore	6.6.2	Swagger/OpenAPI generation and Swagger UI.

11. Other Tools and Technologies

- Visual Studio / .NET development environment - project includes Visual Studio solution metadata and launch settings.
- Git - the submitted project includes Git repository metadata and branch history.
- GitHub - source-control collaboration/submission platform for the project repository.
- Swagger UI - interactive backend endpoint testing during development.
- Browser Fetch API - JavaScript HTTP client used by the frontend.
- ASP.NET Core static-file middleware - serves the HTML, CSS and JavaScript frontend from wwwroot.
- ASP.NET Core IFormFile and .NET file streams - used for CV upload and protected file storage.
- Entity Framework Core migrations - database schema creation and evolution.

12. Technical Stack Summary

Category	Final Project Technology
.NET	.NET 8.0 (net8.0)
Frontend	HTML5, CSS3, Vanilla JavaScript
Backend	ASP.NET Core Web API (C#)
Database	Microsoft SQL Server LocalDB

ORM	Entity Framework Core 8.0.22
Authentication	JWT Bearer Authentication
API Style	RESTful JSON API
API Documentation	Swagger / OpenAPI
Frontend HTTP	Browser Fetch API
File Storage	Protected local server folder outside wwwroot
Matching	Deterministic rule-based weighted C# matching
Source Control	Git / GitHub

13. Verification Note

This technical stack document was prepared from the final submitted project archive. Where earlier planning/task documents may describe a proposed stack, this document reports the technologies and versions present in the final implementation. In particular, the project file confirms TargetFramework net8.0 and package versions shown above, while appsettings.json confirms the SQL Server LocalDB connection used by the final project.